

ส่วนที่ 1: Software Design & Principles Explanation

🧠 1. สถาปัตยกรรมและ GRASP Patterns

ระบบนี้ออกแบบโดยประยุกต์ใช้หลักการ GRASP (General Responsibility Assignment Software Patterns) เพื่อกำหนดหน้าที่ของแต่ละส่วนอย่างชัดเจน:

- **Controller Pattern:** GameController ทำหน้าที่รับ HTTP Requests (GET/POST) จาก Client แปลงข้อมูลพารามิเตอร์และส่งต่อให้ Service ประมวลผล โดยไม่ยุ่งเกี่ยวกับ Business Logic ใดๆ หรือ Database Query
- **High Cohesion:** แต่ละชั้น (Layer) ทำหน้าที่เฉพาะของตนเอง เช่น GameRepository เน้นการสื่อสารกับ Database เท่านั้น ส่วน GameService เน้นจัดการ Business Flow
- **Low Coupling:** ลดการผูกติดกันของ Object โดยใช้ Interface (DiscountStrategy) และการฉีด Dependency Injection ผ่าน Constructor ช่วยให้เปลี่ยน Implementations ได้โดยไม่กระทบ Controller
- **Information Expert:** ให้ Game และ DiscountStrategy แต่ละตัวเป็นผู้จัดการข้อมูลของตนเอง เช่น Strategy แต่ละรูปแบบเป็นผู้รู้สูตรการคำนวณราคาสุทธิที่ถูกต้อง
- **Indirection:** ใช้ GameService เป็นตัวกลางระหว่าง GameController กับ GameRepository และ DiscountContext ช่วยลดการพึ่งพาซึ่งกันและกันโดยตรง

🔗 2. การประยุกต์ใช้ High-Level SOLID Principles

- **Single Responsibility Principle (SRP):** ทุก Class มีหน้าที่เดียว เช่น GameRepository จัดการ persistence logic เท่านั้น, GameController จัดการ routing/view navigation
- **Open/Closed Principle (OCP):** โครงสร้างรองรับการขยายแต่ปิดการแก้ไข หากต้องการเพิ่มโปรโมชั่นส่วนลดใหม่ (เช่น ส่วนลด VIP) สามารถสร้าง Class ใหม่ที่ Implement DiscountStrategy ได้เลยโดยไม่ต้องแก้ไขโค้ดใน Service
- **Liskov Substitution Principle (LSP):** Class ย่อยทั้งหมด (StudentDiscountStrategy, SeasonalSaleStrategy, NoDiscountStrategy) สามารถถูกใช้งานแทนที่ Interface DiscountStrategy ได้อย่างสมบูรณ์โดยไม่ทำให้ระบบผิดพลาด
- **Interface Segregation Principle (ISP):** Interface DiscountStrategy กำหนดเฉพาะ Method ที่จำเป็นต้องใช้งานจริงๆ ไม่บังคับให้ Implement Method ที่ไม่ได้ใช้
- **Dependency Inversion Principle (DIP):** High-level modules (GameService) พึ่งพา Abstraction (DiscountStrategy, GameRepository) แทนที่จะพึ่งพา Concrete Classes โดยตรง

🌱 3. การประยุกต์ใช้ Strategy Pattern

ประยุกต์ใช้ในการคำนวณส่วนลดราคาเกมที่แตกต่างกันตามประเภทโปรโมชั่น:

- **โครงสร้าง:** มี Interface DiscountStrategy และ Implementations ได้แก่ NoDiscountStrategy (ไม่ลดราคา), StudentDiscountStrategy (ลด 10%), และ SeasonalSaleStrategy (ลด 20%)
- **การทำงาน:** DiscountContext จะรับ Strategy ที่ต้องการและเรียกใช้ calculateDiscount(price)
- **ประโยชน์ด้าน OCP:** เมื่อมีนโยบายส่วนลดใหม่เข้ามา Developers เพียงเพิ่ม Class ใหม่ที่ Implement Interface โดยไม่ต้องแตะต้องหรือเสี่ยงทำให้ Logic ส่วนลดเดิมพัง

🏠 4. Layered Architecture

การแยก Service Layer ออกจาก Controller และ Repository มีความสำคัญดังนี้:

- **แยก Business Logic ชัดเจน:** Controller ควรทำหน้าที่เพียงจัดการ Request/Response ส่วน Repository ทำงานเฉพาะ CRUD กับ Database การมี Service Layer ทำให้ Business Logic เช่น การคำนวณราคาหลังหักส่วนลดรวมอยู่ในที่เดียว
- **ประโยชน์ด้าน Low Coupling & High Cohesion:** ทำให้สามารถเปลี่ยน Database Framework (เช่น จาก JPA ไปเป็น JDBC) หรือเปลี่ยนรูปแบบ UI (จาก Thymeleaf ไปเป็น REST API JSON) ได้ โดยไม่ต้องแก้ไข Business Logic ใน Service Layer เลย

🔄 5. Execution Flow (ลำดับการทำงานของระบบ)

1. **Request Ingest:** Browser ส่ง HTTP Request (เช่น POST /games/add) มายัง GameController
2. **Controller Handling:** GameController รับ Model Object (Game) ที่ Binding มาจาก Form
3. **Service Delegation:** Controller เรียกใช้ GameService.saveGame(game)
4. **Strategy Execution:** ภายใน Service หรือ Entity จะอ้างอิง DiscountContext เพื่อเลือก DiscountStrategy (เช่น STUDENT -> StudentDiscountStrategy) และคำนวณ finalPrice
5. **Database Persistence:** GameService เรียก GameRepository.save(game) ซึ่ง Spring Data JPA จะส่ง SQL INSERT ไปยัง PostgreSQL
6. **Response & Rendering:** Controller ส่งผลลัพธ์พร้อมค่า Model กลับไปแสดงผลที่ Thymeleaf HTML View ส่ง HTML บล็อกกลับไปแสดงหน้าจอ Browser

ส่วนที่ 2: Code Implementation & Explanation

1. Entity Layer (Game.java)

ทำหน้าที่เป็น Domain Model ที่ Mapping เข้ากับตาราง games ในฐานข้อมูล PostgreSQL ผ่าน Jakarta Persistence (JPA) Annotation มีฟิลด์ @Transient เพื่อเก็บค่า finalPrice ที่คำนวณจาก Strategy ในหน่วยความจำโดยไม่ต้องบันทึกลง Database

```
src > main > java > com > example > lab7 > model > J Game.java > Game > finalPrice
1  package com.example.lab7.model;
2
3  import jakarta.persistence.*;
4  import java.time.LocalDate;
5
6  @Entity
7  @Table(name = "games")
8  public class Game {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     private String title;
15     private String genre;
16     private String platform;
17     private Double rating;
18     private Double price;
19     private String discountType; // "NONE", "STUDENT", "SEASONAL"
20     private LocalDate releaseDate;
21
22     @Transient
23     private Double finalPrice;
24
25     public Game() {}
26
27     public Game(String title, String genre, String platform, Double rating, Double price, String discountType, LocalDate releaseDate) {
28         this.title = title;
29         this.genre = genre;
30         this.platform = platform;
31         this.rating = rating;
32         this.price = price;
33         this.discountType = discountType;
34         this.releaseDate = releaseDate;
35     }
36
37     // --- Helper Methods สำหรับ Thymeleaf ---
38
39     // แปลงประเภทส่วนลดเป็นชื่อภาษาไทยที่ list.html เรียกใช้
40     public String getDiscountName() {
41         if ("STUDENT".equalsIgnoreCase(discountType)) return "ส่วนลดนักศึกษา (10%)";
42         if ("SEASONAL".equalsIgnoreCase(discountType)) return "ส่วนลดเทศกาล (20%)";
43         return "ราคาปกติ";
44     }
45
46     // คำนวณราคาหลังหักส่วนลดเพื่อป้องกัน NullPointerException
47     public Double getFinalPrice() {
48         if (finalPrice != null) return finalPrice;
49         if (price == null) return 0.0;
50
51         if ("STUDENT".equalsIgnoreCase(discountType)) return price * 0.9;
52         if ("SEASONAL".equalsIgnoreCase(discountType)) return price * 0.8;
53         return price;
54     }
55
56     // --- Getters & Setters ---
57     public Long getId() { return id; }
58     public void setId(Long id) { this.id = id; }
59
60     public String getTitle() { return title; }
61     public void setTitle(String title) { this.title = title; }
62
63     public String getGenre() { return genre; }
64     public void setGenre(String genre) { this.genre = genre; }
65
66     public String getPlatform() { return platform; }
67     public void setPlatform(String platform) { this.platform = platform; }
68
69     public Double getRating() { return rating; }
70     public void setRating(Double rating) { this.rating = rating; }
71
72     public Double getPrice() { return price; }
73     public void setPrice(Double price) { this.price = price; }
74
75     public String getDiscountType() { return discountType; }
76     public void setDiscountType(String discountType) { this.discountType = discountType; }
77
78     public LocalDate getReleaseDate() { return releaseDate; }
79     public void setReleaseDate(LocalDate releaseDate) { this.releaseDate = releaseDate; }
80
81     public void setFinalPrice(Double finalPrice) { this.finalPrice = finalPrice; }
82 }
```

2. Repository Layer (GameRepository.java)

ทำหน้าที่เป็น Data Access Layer สื่อสารโดยตรงกับ PostgreSQL โดยสืบทอดจาก JpaRepository ทำให้สามารถใช้งานคำสั่งจัดการฐานข้อมูลขั้นพื้นฐาน (CRUD: save, findAll, findById, deleteById) ได้ทันทีโดยไม่ต้องเขียนคำสั่ง SQL เอง

```
J Game.java J DiscountStrategy.java J SeasonalSaleStrategy.java J GameRepository.java 1 X # style.css add.html
src > main > java > com > example > lab7 > repository > J GameRepository.java > ...
1 package com.example.lab7.repository;
2
3 import com.example.lab7.model.Game;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface GameRepository extends JpaRepository<Game, Long> {
9 }
```

3. Strategy Package (Discount Strategy)

กลุ่มคลาสสำหรับคำนวณส่วนลดราคาเกมตามหลัก Strategy Pattern เพื่อสนับสนุน Open/Closed Principle (OCP)

3.1 Interface (DiscountStrategy.java)

กำหนดพฤติกรรมกลางสำหรับการคำนวณส่วนลด

```
J Game.java J DiscountStrategy.java X J SeasonalSaleStrategy.java J GameRepository.java 1 # style.css add.html
src > main > java > com > example > lab7 > strategy > J DiscountStrategy.java > DiscountStrategy
1 package com.example.lab7.strategy;
2
3 public interface DiscountStrategy {
4     double calculateFinalPrice(double originalPrice);
5     String getStrategyName();
6 }
```

3.2 Implementations

สร้างกลยุทธ์ส่วนลดแบบต่างๆ และลงทะเบียนเป็น Spring Bean ด้วย @Component("NAME") เพื่อให้อ้างอิงชื่อ Bean ได้ง่าย

```
ameRepository.java 1 # style.css add.html J GameService.java J NoDiscountStrategy.java
src > main > java > com > example > lab7 > strategy > J NoDiscountStrategy.java > NoDiscountStrategy
1 package com.example.lab7.strategy;
2
3 public class NoDiscountStrategy implements DiscountStrategy {
4     @Override
5     public double calculateFinalPrice(double originalPrice) {
6         return originalPrice;
7     }
8
9     @Override
10    public String getStrategyName() {
11        return "ราคาปกติ";
12    }
13 }

src > main > java > com > example > lab7 > strategy > J StudentDiscountStrategy.java > StudentDiscountStrategy
1 package com.example.lab7.strategy;
2
3 public class StudentDiscountStrategy implements DiscountStrategy {
4     @Override
5     public double calculateFinalPrice(double originalPrice) {
6         return originalPrice * 0.90; // ลด 10%
7     }
8
9     @Override
10    public String getStrategyName() {
11        return "ส่วนลดนักศึกษา 10%";
12    }
13 }

J SeasonalSaleStrategy.java X J DiscountContext.java J GameRepository J GameService.java J GameController.java
src > main > java > com > example > lab7 > strategy > J SeasonalSaleStrategy.java > SeasonalSaleStrategy
1 package com.example.lab7.strategy;
2
3 public class SeasonalSaleStrategy implements DiscountStrategy {
4     @Override
5     public double calculateFinalPrice(double originalPrice) {
6         return originalPrice * 0.80; // ลด 20%
7     }
8
9     @Override
10    public String getStrategyName() {
11        return "ส่วนลดเทศกาล 20%";
12    }
13 }
```

3.3 Strategy Context (DiscountContext.java)

ทำหน้าที่บริหารจัดการและเลือกใช้ Strategy ให้ตรงตามประเภทที่ส่งเข้ามา โดยประยุกต์ใช้ Constructor Injection ในการรวบรวม Bean ของ DiscountStrategy ทั้งหมดเข้ามาเก็บใน Map<String, DiscountStrategy> อัตโนมัติ

```
src > main > java > com > example > lab7 > strategy > J DiscountContext.java > DiscountContext
1  package com.example.lab7.strategy;
2
3  public class DiscountContext {
4      private DiscountStrategy strategy;
5
6      public DiscountContext(String discountType) {
7          if (discountType == null) {
8              this.strategy = new NoDiscountStrategy();
9              return;
10         }
11
12         switch (discountType.toUpperCase()) {
13             case "STUDENT":
14                 this.strategy = new StudentDiscountStrategy();
15                 break;
16             case "SEASONAL":
17                 this.strategy = new SeasonalSaleStrategy();
18                 break;
19             case "NONE":
20             default:
21                 this.strategy = new NoDiscountStrategy();
22                 break;
23         }
24     }
25
26     public double executeStrategy(double price) {
27         return strategy.calculateFinalPrice(price);
28     }
29
30     public String getStrategyName() {
31         return strategy.getStrategyName();
32     }
33 }
```

4. Service Layer (GameService.java)

ศูนย์กลางการทำงานของ Business Logic ทำหน้าที่เชื่อมโยงระหว่าง Controller, Repository และ Strategy Context การอธิบาย Constructor Injection ใน Service Layer:

GameService ประกาศตัวแปร gameRepository และ discountContext เป็น private final เพื่อรับประกันว่า Dependency เหล่านี้จะไม่ถูกเปลี่ยนแปลงหลังจาก Object ถูกสร้างขึ้น และรับ Dependency เหล่านั้นผ่าน Constructor Injection แทนการใช้ @Autowired ที่ตัวแปรโดยตรง ช่วยลดการผูกติดกับ Spring IoC Container เพียงๆ (Low Coupling) และทำให้เขียน Unit Test โดย Mock Object ใส่เข้ามาได้ง่ายขึ้น

```
src > main > java > com > example > lab7 > service > J GameService.java > Language Support for Java(TM) by Red Hat > GameService > applyDiscount(Game)
1  package com.example.lab7.service;
2
3  import com.example.lab7.model.Game;
4  import com.example.lab7.repository.GameRepository;
5  import com.example.lab7.strategy.DiscountContext;
6  import org.springframework.stereotype.Service;
7
8  import java.util.List;
9  import java.util.Optional;
10
11  @Service
12  public class GameService {
13
14      private final GameRepository gameRepository;
15
16      // Constructor Injection
17      public GameService(GameRepository gameRepository) {
18          this.gameRepository = gameRepository;
19      }
20
21      public List<Game> getAllGames() {
22          List<Game> games = gameRepository.findAll();
23          for (Game game : games) {
24              applyDiscount(game);
25          }
26          return games;
27      }
28
29      public Optional<Game> getGameById(Long id) {
30          Optional<Game> game = gameRepository.findById(id);
31          game.ifPresent(this::applyDiscount);
32          return game;
33      }
34
35      public Game saveGame(Game game) {
36          return gameRepository.save(game);
37      }
38
39      public void deleteGame(Long id) {
40          gameRepository.deleteById(id);
41      }
42
43      private void applyDiscount(Game game) {
44          if (game.getPrice() != null) {
45              DiscountContext context = new DiscountContext(game.getDiscountType());
46              double finalPrice = context.executeStrategy(game.getPrice());
47              game.setFinalPrice(finalPrice);
48          }
49      }
50  }
```

5. Controller Layer (GameController.java)

ทำหน้าที่เป็น Controller ตามแนวคิด GRASP Controller Pattern คอยจัดการ HTTP Request, บรรจุข้อมูลใส่ Model และเลือกส่ง Template Page (Thymeleaf HTML) กลับไปยัง Client Browser

💡 การอธิบาย Constructor Injection ใน Controller Layer:

GameController จะไม่ทำการสร้าง instance ของ GameService ขึ้นมาเองด้วยคำสั่ง new แต่ใช้วิธีเรียกใช้ผ่าน Constructor Injection โดย Spring Framework จะจัดการค้นหาและส่ง GameService Bean ที่พร้อมใช้งานเข้ามาผ่านทาง Constructor ของ Controller ช่วยรองรับหลัก Dependency Inversion Principle (DIP) ทำให้ Controller สนใจเฉพาะการควบคุม Flow หน้าจอ โดยผลักภาระงานประมวลผลทั้งหมดไปให้ Service Layer

```
src > main > java > com > example > lab7 > controller > J GameController.java > Language Support for Java(TM) by Red Hat > GameController

1  package com.example.lab7.controller;
2
3  import com.example.lab7.model.Game;
4  import com.example.lab7.service.GameService;
5  import org.springframework.stereotype.Controller;
6  import org.springframework.ui.Model;
7  import org.springframework.web.bind.annotation.*;
8
9  @Controller
10 @RequestMapping("/games")
11 public class GameController {
12
13     private final GameService gameService;
14
15     // Constructor Injection
16     public GameController(GameService gameService) {
17         this.gameService = gameService;
18     }
19
20     // READ All
21     @GetMapping
22     public String listGames(Model model) {
23         model.addAttribute("games", gameService.getAllGames());
24         return "games/list";
25     }
26
27     // CREATE Form
28     @GetMapping("/add")
29     public String showAddForm(Model model) {
30         model.addAttribute("game", new Game());
31         return "games/add";
32     }
33
34     // CREATE Save
35     @PostMapping("/add")
36     public String saveGame(@ModelAttribute("game") Game game) {
37         gameService.saveGame(game);
38         return "redirect:/games";
39     }
40
41     // UPDATE Form
42     @GetMapping("/edit/{id}")
43     public String showEditForm(@PathVariable("id") Long id, Model model) {
44         Game game = gameService.getGameById(id)
45             .orElseThrow(() -> new IllegalArgumentException("Invalid game Id:" + id));
46         model.addAttribute("game", game);
47         return "games/edit";
48     }
49
50     // UPDATE Save
51     @PostMapping("/edit/{id}")
52     public String updateGame(@PathVariable("id") Long id, @ModelAttribute("game") Game game) {
53         game.setId(id);
54         gameService.saveGame(game);
55         return "redirect:/games";
56     }
57
58     // DELETE Form/Confirm
59     @GetMapping("/delete/{id}")
60     public String showDeleteConfirm(@PathVariable("id") Long id, Model model) {
61         Game game = gameService.getGameById(id)
62             .orElseThrow(() -> new IllegalArgumentException("Invalid game Id:" + id));
63         model.addAttribute("game", game);
64         return "games/delete";
65     }
66
67     // DELETE Action
68     @PostMapping("/delete/{id}")
69     public String deleteGame(@PathVariable("id") Long id) {
70         gameService.deleteGame(id);
71         return "redirect:/games";
72     }
73 }
```

Game Catalog

CP353002 — Lab 7

← กลับไปรายการเกม

+ เพิ่มเกมใหม่

กรอกข้อมูลเกมเพื่อทำการเพิ่มลงในระบบ

↓ ชื่อเกม

No time to Relax (673380057-5 SEC 1)

🎮 แนวเกม (Genre)

📺 แพลตฟอร์ม (Platform)

FamilyPC

★ คะแนน (Rating)

📅 ราคาคงที่ (บาท)

% ส่วนลด (Strategy)

10150ส่วนลดนักศึกษา (10%)

📅 วันวางจำหน่าย (Release Date)

09/07/2026

📁

📌 บันทึก

🔄 ยกเลิก

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

📁 รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

➕ เพิ่มเกมใหม่

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาคงที่ (บาท)	จัดการ
1	Shin Godzilla	Kaijus	PC	★ 10.0	2026-08-07	ราคาปกติ	1500.00	📌 🗑️
2	No time to Relax (673380057-5 SEC 1)	Family	PC	★ 10.0	2026-09-07	ส่วนลดนักศึกษา (10%)	135.00 150.00	📌 🗑️

📁 ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

Game Catalog

CP353002 — Lab 7

← กลับไปรายการเกม

✎ แก้ไขข้อมูลเกม

แก้ไขข้อมูลเกม: No time to Relax (673380057-5 SEC 1)

↓ ชื่อเกม

No time to Relax (673380057-5 SEC 1)

🎮 แนวเกม (Genre)

📺 แพลตฟอร์ม (Platform)

Family/ChillPC

★ คะแนน (Rating)

📅 ราคาคงที่ (บาท)

% ส่วนลด (Strategy)

10.0200ส่วนลดเทศกาล (20%)

📅 วันวางจำหน่าย (Release Date)

08/15/2026

📁

📌 บันทึก

🔄 ยกเลิก

